Reflexión individual -Saul Gabriel Nolasco Cen

En el desarrollo de software y en el análisis de sistemas, el rendimiento de una aplicación no depende únicamente de la capacidad del hardware, sino de la eficiencia con la que se eligen las estructuras de datos y los algoritmos para procesar la información. En esta situación problema trabajamos con el archivo bitacora.txt, el cual contiene un conjunto masivo de registros donde cada entrada almacena una dirección IP, la fecha, la hora y un mensaje de error. Dado que al iniciar el programa no conocemos la cantidad exacta de líneas a procesar, usar una estructura estática resulta inviable, ya que nos obligaría a reservar espacio de manera arbitraria corriendo el riesgo de sufrir un desbordamiento de memoria o malgastar recursos. La implementación de una lista doblemente ligada resuelve este inconveniente mediante la asignación dinámica de memoria en el *heap*, creando nodos de forma flexible a medida que se lee cada registro.

Para dimensionar el impacto real de la eficiencia algorítmica en este problema, consideremos como referencia técnica que una computadora promedio procesa alrededor de 10^8 operaciones básicas por segundo. Si tomamos una bitácora de tamaño considerable, por ejemplo con $262{,}144$ registros, e intentáramos ordenarla utilizando un algoritmo elemental como Bubble Sort cuya complejidad en el peor de los casos es $O(N^2)$, el procesador tendría que ejecutar aproximadamente $(262{,}144)^2 \approx 6.87 \times 10^{10}$ operaciones. Esto implicaría una espera de casi 11.4 minutos únicamente para ordenar la bitácora antes de poder realizar cualquier consulta. En cambio, al adaptar Merge Sort para trabajar sobre los punteros de la lista doblemente ligada, logramos reducir la complejidad temporal a $O(N \log_2 N)$. Para esos $262{,}144$ registros, $\log_2(262{,}144) = 18$, lo que significa realizar $262{,}144 \times 18 \approx 4.71 \times 10^6$ operaciones. Esto reduce el tiempo de procesamiento a solo 0.047 segundos, convirtiendo un proceso que habría congelado la ejecución por minutos en una tarea prácticamente instantánea.

Adicionalmente, el problema nos exige ordenar la información bajo tres criterios jerárquicos: primero por la IP (realizando una conversión numérica por octetos para evitar fallos del orden alfabético donde "10.50.100.1" se colocaría erróneamente antes que "10.50.2.1"), en caso de empate por fecha y hora, y finalmente por el mensaje de error. Una vez estructurada la lista, debemos cumplir dos requerimientos con direcciones opuestas: guardar los datos de forma ascendente en el archivo SortedData.txt y desplegar en pantalla las IPs del rango solicitado por el usuario en orden descendente. Aquí es donde la lista doblemente ligada demuestra su mayor ventaja sobre una lista simplemente ligada. En una lista simple, para mostrar los registros en sentido inverso (de diciembre hacia meses anteriores) habríamos necesitado vaciar los datos en una pila auxiliar ($O(N)$ en memoria extra) o realizar llamadas recursivas costosas. Con la lista doble, gracias al

puntero prev, basta con ubicar el nodo superior del rango consultado y recorrer la información hacia atrás, logrando un despliegue en tiempo $O(k)$ proporcional solo a los datos mostrados, sin duplicar memoria.

Sin embargo, para evaluar la eficiencia de forma completa, también debemos identificar las limitaciones inherentes a la lista doblemente ligada. Aunque los datos estén perfectamente ordenados, en una lista no es posible aplicar una búsqueda binaria tradicional en $O(\log_2 N)$ como se haría en un arreglo contiguo, ya que la lista carece de acceso aleatorio directo por índice ($O(1)$). Para encontrar las IPs de inicio y fin ingresadas por el usuario, el programa está obligado a hacer un recorrido secuencial nodo por nodo en $O(N)$, además de lidiar con el consumo extra de 16 bytes por nodo debido a los dos punteros (next y prev) y los fallos de caché por la dispersión de memoria en el *heap*. En conclusión, esta actividad demuestra que la lista doblemente ligada es una herramienta ideal para resolver este problema por la versatilidad de su navegación bidireccional, pero también nos enseña a ponderar los *trade-offs* entre facilidad de iteración, uso de memoria y velocidad de búsqueda en aplicaciones reales.